

Guía Completa de Alineación y Migración del MER de Draw.io hacia el Proyecto Django (Monagua)

Esta guía detalla el análisis comparativo exhaustivo y los pasos de desarrollo requeridos para que el modelo de datos del proyecto Django (MNG_WEB) refleje de manera **100% exacta y fiel** el Modelo Entidad-Relación (MER) principal diseñado en Draw.io (Diagrama sin título.drawio.html), transformando el estado actual reflejado en modelo_datos.png.

Resumen Ejecutivo del Diagnóstico

Tras comparar las **21 entidades** de Diagrama sin título.drawio.html contra los **18 modelos** actuales representados en modelo_datos.png y el código fuente de las aplicaciones Django (usuarios, catalogo, reservas, pagos, promociones, comunidad, notificaciones), se identifican tres niveles de intervención:

1. **5 Tablas Completamente Nuevas** que no existen en el código actual de Django:
 2. PlanGuia (Asignación y programación de guías turísticos por paquete)
 3. PaquetePromocion (Tabla intermedia M2M entre Paquete, Promoción y Tarifa con precios promocionales)
 4. Factura (Facturación de reservas y comprobantes de pago)
 5. SeguroViaje (Gestión de pólizas y coberturas de seguro asociadas a reservas)
- Poliza (Catálogo de aseguradoras y pólizas de seguro)

Reestructuración de Claves Foráneas (FK) Críticas:

8. **Reserva:** Actualmente apunta a Usuario (usuario_id). Debe reorientarse para apuntar directamente a Cliente (codigo_cliente).
9. **Promocion:** Romper la relación 1:N directa con Paquete e implementar la relación N:M a través de PaquetePromocion.
10. **Blog:** Incorporar la FK codigo_usuario hacia Usuario.

Actividades / PaqueteActividad: Trasladar el campo nivel_dificultad desde Actividades hacia la tabla intermedia PaqueteActividad (dificultad_nivel).

Homologación de Nombres de Campos y Tipos de Datos:

13. Ajustar nombres de atributos en GuiaTuristico, Cancelacion, Calificacion, ComprobantePago -> Pago, Blog, etc., garantizando consistencia semántica.

User Review Required

IMPORTANTE: > ****Cambios de Rompimiento (Breaking Changes) en Modelos y Base de Datos****: > 1. ****Estructura de la Base de Datos****: Se recomienda reiniciar y volver a migrar la base de datos local (db.sqlite3) ejecutando python

reset_db.py` y `python poblar_bd.py` adaptado, para asegurar que los scripts de prueba carguen datos válidos con la nueva arquitectura del MER. > 2. **\*\*Refactorización de Claves Foráneas (`Reserva.usuario` -> `Reserva.cliente`)\*\*:** Las vistas, formularios y plantillas HTML que accedan a `reserva.usuario` deberán ser actualizados para navegar a través de `reserva.cliente.usuario` o `reserva.cliente`. > 3. **\*\*Compatibilidad con `AbstractUser` de Django\*\*:** El MER de Draw.io separa los atributos del usuario genérico (`usuario`) y el perfil del cliente (`cliente`). En Django, la buena práctica es mantener `Usuario(AbstractUser)` para la autenticación (que mapea `email`, `password`, `is_active`, `date_joined`) y extender la información específica del cliente en `Cliente`.

Open Questions

NOTA: > ****1. Manejo de Ubicación en `Cliente`**:** > El modelo `Cliente` en Django actualmente contiene los campos `pais`, `departamento` y `ciudad`. En el diagrama de Draw.io, estos 3 campos no están en la tabla `cliente`, pero `cliente` incluye `primer\_nombre`, `segundo\_nombre`, `primer\_apellido`, `segundo\_apellido`, `tipo\_documento`, `numero\_documento` y `telefono\_contacto`. ¿Deseas conservar `pais`, `departamento` y `ciudad` como información complementaria o eliminarlos para ceñirnos estrictamente a Draw.io? > > ****2. Mantenimiento de `imagen` en `Paquete` y `ComprobantePago`**:** > Los modelos actuales de Django incluyen `imagen` en `Paquete` y `ComprobantePago`. Aunque Draw.io lista campos conceptuales como `imagen\_comprobante`, mantendremos los tipos `ImageField` de Django para dar soporte a la subida de archivos multimedia en la plataforma web.

Matriz de Homologación de Entidades (Draw.io vs Django Actual)

Entidad MER (Draw.io)	Modelo Django Actual	Estado / Acción Requerida
usuario	usuarios.Usuario	Modificar: Mantener AbstractUser. Mapear correo_electronico -> email, contraseña_usuario -> password, estado -> is_active.
cliente	usuarios.Cliente	Modificar: Agregar primer_nombre, segundo_nombre, primer_apellido, segundo_apellido, tipo_documento, numero_documento, telefono_contacto.
guia_turistico	usuarios.GuiaTuristico	Modificar: Agregar entidad_salud, estado. Renombrar/Alias: licencia_turismo -> numero_tarjeta_profesional, biografia -> descripcion_experiencia.

plan_guia	<i>Inexistente</i>	NUEVO MODELO: Crear PlanGuia con FK a GuiaTuristico y FK a Paquete.
paquete	catalogo.Paquete	Modificar: Mantener campos core (nombre, descripcion, dias_duracion, noches_duracion, punto_encuentro, hora_encuentro, estado, categoria).
actividad	catalogo.Actividades	Modificar: Mover nivel_dificultad a PaqueteActividad. Renombrar recomendacion_salud -> recomendaciones, apto_para_menores -> apto_menores.
paquete_actividad	catalogo.PaqueteActividad	Modificar: Agregar campo dificultad_nivel.
categoria	catalogo.Categoria	Alineado 100%: nombre, descripcion, estado.
temporada	catalogo.Temporada	Modificar: Agregar campo descripcion (TextField).
tarifa	catalogo.Tarifa	Alineado 100%: FK paquete, FK temporada, precio_adulto, precio_menor, estado.
promoción	promociones.Promocion	Modificar: Retirar FK directa paquete. Agregar nombre_promocion, porcentaje_descuento, fecha_inicio_promocion, fecha_fin_promocion, condiciones, codigo_cupon, estado.
paquete_promociones	<i>Inexistente</i>	NUEVO MODELO: Crear PaquetePromocion (FK paquete, FK tarifa, FK promocion, valor_adulto_condescuento, valor_menor_condescuento).
reserva	reservas.Reserva	Modificar: Cambiar FK usuario por FK cliente (Cliente). Renombrar fecha -> fecha_inicio, estado -> estado_reserva.

cancelación	<code>reservas.Cancelacion</code>	Modificar: Agregar <code>fecha_reembolso</code> , <code>valor_reembolsado</code> , <code>imagen_comprobante</code> .
seguro_viaje	<i>Inexistente</i>	NUEVO MODELO: Crear SeguroViaje (FK reserva, <code>codigo_poliza</code> , <code>numero_poliza</code> , <code>cobertura_detalle</code> , <code>costo_seguro</code>).
poliza	<i>Inexistente</i>	NUEVO MODELO: Crear Poliza (<code>nombre_aseguradora</code> , <code>descripcion</code> , <code>estado</code>).
pago	<code>pagos.ComprobantePago</code>	Modificar/Refactor: Homologar a Pago / ComprobantePago con <code>metodo_pago</code> , <code>fecha_pago</code> , <code>imagen_comprobante</code> , <code>estado_transaccion</code> , FK reserva.
factura	<i>Inexistente</i>	NUEVO MODELO: Crear Factura (FK reserva, FK pago, <code>fecha_emision</code> , <code>estado</code> , <code>valor_subtotal</code> , <code>valor_total</code>).
pqrs	<code>comunidad.PQRS</code>	Alineado: FK cliente, tipo, asunto, <code>descripcion</code> , <code>estado</code> , fecha.
seguimiento	<code>comunidad.Historial</code>	Modificar/Alias: Historial actúa como Seguimiento (FK pqrs, <code>fecha_respuesta</code> , <code>respuesta</code>).
calificación	<code>comunidad.Calificacion</code>	Modificar: Renombrar <code>puntaje</code> -> <code>puntaje_estrellas</code> , <code>fecha</code> -> <code>fecha_calificacion</code> .
blog	<code>comunidad.Blog</code>	Modificar: Agregar FK usuario -> Usuario. Renombrar <code>imagen</code> -> <code>imagen_destacada</code> , <code>publicado</code> -> <code>estado</code> .
auditoria	<code>notificaciones.Auditoria</code>	Alineado: <code>acciones_realizada</code> , <code>tabla_afectada</code> , fecha, hora, <code>observacion</code> , <code>valor_anterior</code> , <code>nuevo_valor</code> , FK <code>codigo_usuario</code> .

Plan Detallado de Cambios por Aplicación Django

Aplicación: **usuarios**

[models.py](#)

[MODIFY] [models.py](#)

1. **Usuario:**
2. Conservar como heredero de `AbstractUser`.
3. Garantizar correspondencia de email (`correo_electronico`), password (`contraseña_usuario`), `is_active` (`estado`), `date_joined` (`fecha_registro`).
4. **Cliente:**
5. Agregar campos de identificación personal especificados en Draw.io:
 - `primer_nombre = models.CharField(max_length=50)`
 - `segundo_nombre = models.CharField(max_length=50, blank=True, null=True)`
 - `primer_apellido = models.CharField(max_length=50)`
 - `segundo_apellido = models.CharField(max_length=50, blank=True, null=True)`
 - `tipo_documento = models.CharField(max_length=20, choices=Usuario.TipoDocumento.choices)`
 - `numero_documento = models.CharField(max_length=20, unique=True)`
 - `telefono_contacto = models.CharField(max_length=15)`
6. **GuiaTuristico:**
7. Renombrar / agregar campos:
 - `numero_tarjeta_profesional = models.CharField(max_length=50)` (sustituye/alias de `licencia_turismo`)
 - `descripcion_experiencia = models.TextField()` (sustituye/alias de `biografia`)
 - `experiencia_fecha = models.DateField(null=True, blank=True)`
 - `entidad_salud = models.CharField(max_length=100, blank=True, null=True)`
 - `estado = models.BooleanField(default=True)`
8. **[NUEVO MODELO] PlanGuia:**
9. Implementar el modelo `PlanGuia`:

```
python class PlanGuia(models.Model):  
    guia_turistico = models.ForeignKey(GuiaTuristico, on_delete=models.CASCADE, related_name='planes_guia')  
    paquete = models.ForeignKey('catalogo.Paquete', on_delete=models.CASCADE, related_name='planes_guia')  
    idioma_servicio = models.CharField(max_length=50, verbose_name='Idioma del Servicio')  
    fecha_creacion = models.DateTimeField(auto_now_add=True)  
    fecha_inicio_plan = models.DateField()  
    fecha_fin_plan = models.DateField()  
    estado = models.BooleanField(default=True)
```

Aplicación: catalogo

[models.py](#)

[MODIFY] [models.py](#)

1. **Temporada:**
2. Agregar `descripcion = models.TextField(blank=True, null=True, verbose_name='Descripción')`.
3. **Actividades:**
4. Renombrar `recomendacion_salud` -> `recomendaciones`.
5. Renombrar `apto_para_menores` -> `apto_menores`.
6. Remover `nivel_dificultad` de Actividades (se traslada a `PaqueteActividad`).
7. **PaqueteActividad:**
8. Agregar campo `dificultad_nivel = models.CharField(max_length=20, choices=Actividades.NIVEL_CHOICES, default='Media')`.

Aplicación: promociones

[models.py](#)

[MODIFY] [models.py](#)

1. **Promocion:**
2. Eliminar paquete (ForeignKey directa).
3. Homologar nombres de campos con Draw.io:
 - `nombre_promocion = models.CharField(max_length=150)`
 - `porcentaje_descuento = models.PositiveIntegerField()`
 - `fecha_inicio_promocion = models.DateField()`
 - `fecha_fin_promocion = models.DateField()`
 - `condiciones = models.TextField(blank=True, null=True)`
 - `codigo_cupon = models.CharField(max_length=50, blank=True, null=True)`
 - `estado = models.BooleanField(default=True)`
4. **[NUEVO MODELO] PaquetePromocion:**
5. Crear el modelo para respaldar la relación N:M:

```
python class PaquetePromocion(models.Model):
    paquete = models.ForeignKey('catalogo.Paquete', on_delete=models.CASCADE,
    related_name='paquete_promociones')
    tarifa = models.ForeignKey('catalogo.Tarifa',
    on_delete=models.CASCADE, related_name='paquete_promociones')
    promocion = models.ForeignKey(Promocion, on_delete=models.CASCADE, related_name='paquete_promociones')
    valor_adulto_condescuento = models.IntegerField()
    valor_menor_condescuento =
```

models.IntegerField()

Aplicación: reservas

[models.py](#)

[MODIFY] [models.py](#)

1. Reserva:

Modificar la relación usuario:

- Cambiar de `ForeignKey(settings.AUTH_USER_MODEL)` a `ForeignKey('usuarios.Cliente', on_delete=models.CASCADE, related_name='reservas')`.

3. Renombrar `fecha` -> `fecha_inicio`.

4. Renombrar `estado` -> `estado_reserva`.

5. Cancelacion:

6. Agregar campos de Draw.io:

- `fecha = models.DateField(auto_now_add=True)`
- `fecha_reembolso = models.DateField(null=True, blank=True)`
- `valor_reembolsado = models.IntegerField(default=0)`
- `imagen_comprobante = models.ImageField(upload_to='cancelaciones/', null=True, blank=True)`

7. [NUEVO MODELO] Poliza:

8. Implementar el catálogo de pólizas: `python class Poliza(models.Model): nombre_aseguradora = models.CharField(max_length=100) descripcion = models.TextField() estado = models.BooleanField(default=True)`

9. [NUEVO MODELO] SeguroViaje:

10. Implementar el seguro de viaje vinculado a la reserva: `python class SeguroViaje(models.Model): reserva = models.ForeignKey(Reserva, on_delete=models.CASCADE, related_name='seguros_viaje') poliza = models.ForeignKey(Poliza, on_delete=models.CASCADE, related_name='seguros_emitidos') numero_poliza = models.CharField(max_length=50, unique=True) cobertura_detalle = models.TextField() costo_seguro = models.IntegerField()`

Aplicación: pagos

[models.py](#)

[MODIFY] [models.py](#)

1. ComprobantePago / Pago:

Homologar con pago de Draw.io asegurando los campos:

- `metodo_pago = models.CharField(max_length=50)`
- `fecha_pago = models.DateTimeField(auto_now_add=True)`
- `imagen_comprobante = models.ImageField(upload_to='comprobantes/')`
- `estado_transaccion = models.CharField(max_length=20, default='pendiente')`
- `reserva = models.ForeignKey('reservas.Reserva', on_delete=models.CASCADE)`

3. [NUEVO MODELO] Factura:

4. Implementar el modelo Factura:

```
python class Factura(models.Model):  
    reserva = models.ForeignKey('reservas.Reserva', on_delete=models.CASCADE, related_name='facturas')  
    pago = models.ForeignKey(ComprobantePago, on_delete=models.SET_NULL, null=True, blank=True, related_name='facturas')  
    fecha_emision = models.DateTimeField(auto_now_add=True)  
    estado = models.CharField(max_length=20, default='emitida')  
    valor_subtotal = models.DecimalField(max_digits=12, decimal_places=2)  
    valor_total = models.DecimalField(max_digits=12, decimal_places=2)
```

Aplicación: comunidad

[models.py](#)

[MODIFY] [models.py](#)

1. **Calificacion:**
2. Renombrar `puntaje` -> `puntaje_estrellas`.
3. Renombrar `fecha` -> `fecha_calificacion`.
4. **Blog:**
5. Agregar `usuario = models.ForeignKey('usuarios.Usuario', on_delete=models.CASCADE, related_name='blogs')`.
6. Renombrar `imagen` -> `imagen_destacada`.
7. Renombrar `publicado` -> `estado` (`BooleanField(default=True)`).
8. **Historial (Seguimiento):**
9. Mantener con `verbose_name = 'Seguimiento PQRS'` para representar la entidad `seguimiento` de Draw.io.

Aplicación: notificaciones

[models.py](#)

[MODIFY] [models.py](#)

1. **Auditoria:**

2. Asegurar que la relación `codigo_usuario` sea `ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)`.
3. Mantener campos `acciones_realizada`, `tabla_afectada`, `fecha`, `hora`, `observacion`, `valor_anterior`, `nuevo_valor`.

Script de Población y Reconfiguración de Datos

Tras modificar los modelos, los scripts de la raíz del proyecto deberán actualizarse: - `reset_db.py`: Limpiará la base de datos SQLite y reaplicará migraciones (`makemigrations` y `migrate`). - `poblar_bd.py`: Se actualizará para instanciar primero las relaciones obligatorias de `Cliente`, `PaquetePromocion`, `Poliza`, `SeguroViaje`, `PlanGuia` y `Factura`.

Verification Plan

Automated Verification

1. **Compilación y Verificación de Sintaxis de Modelos:** `bash .\venv\Scripts\python.exe manage.py check`
2. **Generación de Migraciones de Django:** `bash .\venv\Scripts\python.exe manage.py makemigrations`
`.\venv\Scripts\python.exe manage.py migrate`
3. **Prueba de Población de Base de Datos:** `bash .\venv\Scripts\python.exe reset_db.py`
`.\venv\Scripts\python.exe poblar_bd.py`

Manual Verification

1. **Generación de Nuevo Diagrama MER con `django-extensions`:** `bash .\venv\Scripts\python.exe manage.py graph_models -a -o nuevo_modelo_datos.png` *Comparar visualmente `nuevo_modelo_datos.png` con `modelo_datos.png` y `Diagrama sin título.drawio.html` para confirmar alineación 1:1.*